

EE-UY/CS-UY 4563: Introduction to Machine Learning

Midterm 1 Solutions, Fall 2018

Prof. Sundeep Rangan

Total of $34 + 36 + 36 = 106$ points. 100 points is considered perfect. So, there are 6 bonus points available.

1. (34 points total)

(a) (10 points) Take,

$$\beta_0 = c_0, \quad \beta_1 = c_1, \quad \beta_2 = c_1 c_2.$$

and

$$\phi_0(x) = 1, \quad \phi_1(x) = \frac{x_1}{1+x_1}, \quad \phi_2(x) = \frac{x_1 x_2}{1+x_1}.$$

Then,

$$f(x_1, x_2) = \sum_{j=0}^2 \beta_j \phi_j(x).$$

(b) (8 points) We invert the equations in part (a):

$$c_0 = \beta_0, \quad c_1 = \beta_1, \quad c_2 = \frac{\beta_2}{c_1} = \frac{\beta_2}{\beta_1}.$$

(c) (8 points) We have

$$\text{RSS}(\beta) = \sum_{i=1}^n (y_i - \hat{y}_i)^2, \quad \hat{y}_i = \sum_{j=0}^{p-1} \beta_j \phi_j(x_{i1}, x_{i2}).$$

We now use chain rule:

$$\frac{\partial \text{RSS}(\beta)}{\partial \beta_k} = 2 \sum_{i=1}^n (\hat{y}_i - y_i) \frac{\partial \hat{y}_i}{\partial \beta_k} = 2 \sum_{i=1}^n (\hat{y}_i - y_i) \phi_k(x_{i1}, x_{i2}).$$

(d) (8 points) One solution is:

```
def fit(X,y):
    n = X.shape[0] # Get number of samples

    # Construct A with the three columns
    col0 = np.ones(n)
    col1 = X[:,0]/(1+X[:,0])
```

```
col2 = X[:,0]*X[:,1]/(1+X[:,0])
A = np.column_stack((col0,col1,col2))

# Solve for beta
beta = lstsq(A,y)

# Get the original parameters
c = np.array([beta[0], beta[1], beta[2]/beta[1]])
```

2. (36 points total)

(a) (10 points) Observe that

$$\max\{0, x - 1\} = \begin{cases} 0 & \text{if } x \leq 1, \\ x - 1 & \text{if } x \geq 1. \end{cases}$$

Therefore,

$$\begin{aligned} f(x, \beta) &= \beta_0 + \beta_1 x + \beta_2 \max\{0, x - 1\} \\ &= \begin{cases} \beta_0 + \beta_1 x & \text{if } x \leq 1, \\ \beta_0 + \beta_1 x + \beta_2(x - 1) & \text{if } x \geq 1 \end{cases} = \begin{cases} \beta_0 + \beta_1 x & \text{if } x \leq 1, \\ (\beta_0 - 1) + (\beta_1 + \beta_2)x & \text{if } x \geq 1. \end{cases} \end{aligned}$$

Now,

$$2|x - 1| = \begin{cases} 2 - 2x & \text{if } x \leq 1, \\ -2 + 2x & \text{if } x \geq 1. \end{cases}$$

Therefore, to get $2|x - 1| = f(x, \beta)$, we can take

$$\beta_0 = 2, \beta_1 = -2, \beta_1 + \beta_2 = 2 \Rightarrow \beta_2 = 4,$$

(b) (8 points) Observe that

$$2|x - 1| \leq 4 \iff |x - 1| \leq 2 \iff x \in [-1, 3].$$

Therefore, since all the training points $x_i \in [0, 2]$,

$$y_i = f_0(x_i) = \min\{4, 2|x_i - 1|\} = 2|x_i - 1|.$$

Hence, the training data matches the function $y = 2|x - 1|$ exactly. Therefore, you will get β from part (a):

$$\hat{\beta} = (2, -2, 4).$$

(c) (8 points) The bias is the difference between the estimated and true function as shown in Fig. 1. At $x = 4$,

$$\begin{aligned} f_0(x) &= \min\{4, 2|x - 1|\} = \min\{4, 6\} = 4, \\ f(x, \beta) &= 2|x - 1| = 2(3) = 6. \end{aligned}$$

Therefore, the bias is,

$$\text{Bias}(x, \hat{\beta}) = f_0(x) - f(x, \hat{\beta}) = 4 - 6 = -2.$$

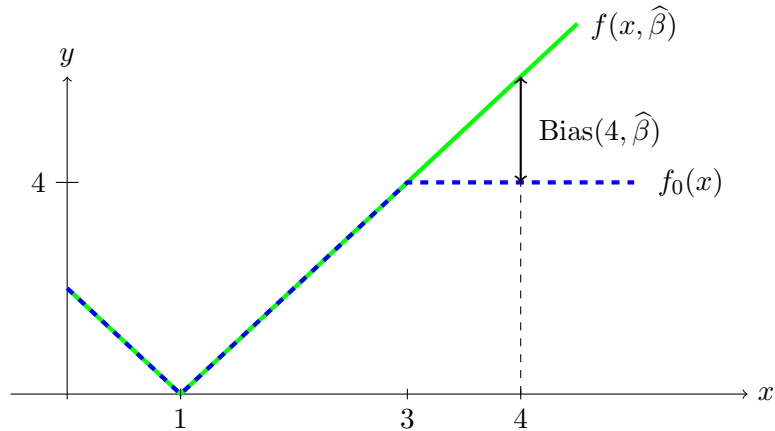


Figure 1: True function (blue) and estimated function (green).

- (d) (5 points for normal rule, 5 points for one SE rule) For the normal rule, we select the model with the lower mean test RSS:

```
rss_mean = np.mean(RSS, axis=1)
if (rss_mean[0] <= rss_mean[1]):
    mod_normal = 0
else
    mod_normal = 1
```

The code below selects the model order for the one SE rule: If the `mod_normal == 0`, we can select `mod_one_se=0`, since the normal rule is already selecting the simplest model. Otherwise, we see if the mean RSS for model 0 is less than mean RSS for model 1 within one SE:

```
if (mod_normal == 0):
    mod_one_se = 0
else:
    # Compute the SE
    K = RSS.shape[1]
    rss_se = np.std(RSS, axis=1)/np.sqrt(K-1)
    rss_tgt = rss_mean[1] + rss_se[1]
    if (rss_mean[0] < rss_tgt):
        mod_one_se = 0
    else
        mod_one_se = 1
```

3. (36 points total)

- (a) (8 points) The four points are shown in Fig. 2.
- (b) (10 points) One way to classify all the points correctly is to use a circular classifier: take $\hat{y} = 1$ if and only if $(x_1, x_2) \in C$, where C is the circle with radius $r = 2$ centered at

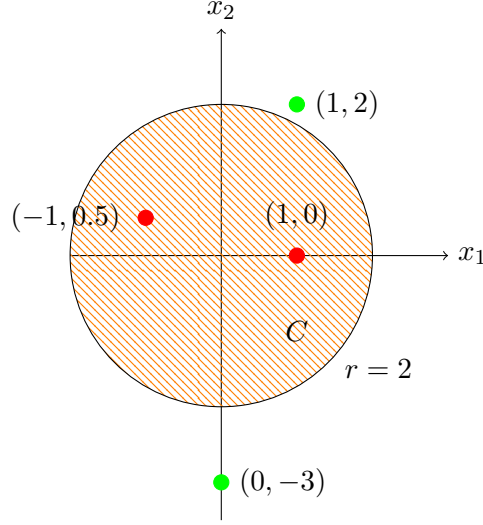


Figure 2: Scatter plot of the data points where the green circles are $y_i = 1$ and the red circles are $y_i = 0$. The hashed area is the region the proposed classifier in part (b) sets $\hat{y} = 1$.

the origin. The circular region is drawn as the orange hashed area in Fig. 2. To put this classifier in the desired form, we note that

$$(x_1, x_2) \in C \iff x_1^2 + x_2^2 \leq 2^2 \iff 4 - x_1^2 - x_2^2 \geq 0.$$

Therefore, we can write the classifier as

$$\hat{y} = \begin{cases} 1 & \text{if } z > 0, \\ 0 & \text{if } z < 0 \end{cases} \quad z = \beta_0 + \beta_1 x_1 + \beta_2 x_1^2 + \beta_3 x_2 + \beta_4 x_2^2,$$

where

$$\beta = (4, 0, -1, 0, -1).$$

- (c) (5 points for (i), 5 points for (ii)) To maximize the probability $P(y = 1|z)$ we want to maximize z . For problem (i), we take the partial derivatives of z with respect to x_1 and x_2 and set them to zero:

$$\begin{aligned} \frac{\partial z}{\partial x_1} = \beta_1 + 2\beta_2 x_1 = 0 &\Rightarrow x_1 = -\frac{\beta_1}{2\beta_2} = \frac{2}{2(3)} = \frac{1}{3} \\ \frac{\partial z}{\partial x_2} = \beta_3 + 2\beta_4 x_2 = 0 &\Rightarrow x_2 = -\frac{\beta_3}{2\beta_4} = \frac{4}{2(5)} = \frac{2}{5}. \end{aligned}$$

For problem (ii), the above calculation will yield a local *minima* of z , not a *maxima*. Indeed, since β_2 and $\beta_4 > 0$, we can take x_1 or $x_2 \rightarrow \infty$ and $z \rightarrow \infty$.

- (d) (8 points) We have

$$P(y_i = 1|x_i) = \frac{1}{1 + e^{-z_i}}, \quad P(y_i = 0|x_i) = \frac{e^{-z_i}}{1 + e^{-z_i}}.$$

We can write this in one formula as:

$$P(y_i = 1|x_i) = \frac{y_i + (1 - y_i)e^{-z_i}}{1 + e^{-z_i}}.$$

The function can then be implemented as follows:

```
def outlier_detect(X,y,beta,t):  
    z = beta[0] + beta[1]*X[:,0] + beta[2]*(X[:,0]**2) +  
        beta[3]*X[:,1] + beta[4]*(X[:,1]**2)  
    py = (y + (1-y)*np.exp(-z))/(1+np.exp(-z))  
    reject = (py < t)  
    return reject
```